

Engineering Deterministic Agent Harnesses: Architecture, Verification Gates, and Non-Autoregressive Decision Primitives for Local Models

The Architectural Imperative: Inversion of Control for Mid-Sized Models

The deployment of small and mid-sized open-weights language models—specifically architectures in the 30B to 35B parameter class such as Qwen 3.5 35B—reveals a structural challenge in autonomous software engineering. While parameters at this scale possess rich latent representations of algorithmic structure, syntax, and API patterns, they exhibit high sensitivity to context pollution, subtle instruction drift during extended agentic loops, and a documented inability to reliably detect their own logical errors through unstructured self-reflection. Autoregressive sampling across long horizons compounds minute stochastic deviations: an unclosed delimiter, an invalid type coercion, or an import of a deprecated interface degrades multi-step tasks into unrecoverable failure loops.

Traditional software engineering has historically maintained codebase integrity through deterministic validation: type checkers, concrete syntax tree parsers, compilers, formatters, and static analyzers. Standard autonomous agent architectures frequently treat these deterministic safeguards merely as optional tools that the model may voluntarily invoke via generic shell interfaces or tool-calling schemas. When an agent must decide autonomously whether to format a diff, run a linter, or parse a compiler trace, 35B-class models often omit the check, hallucinate a passing test run, or become overwhelmed by the sheer volume of unstructured terminal error logs.

Achieving production-grade reliability with mid-sized local models requires an inversion of control. Rather than relying on the language model to act as its own orchestrator and quality inspector, the execution harness must function as an uncompromising, deterministic compiler that wraps the model. In this architecture, the stochastic model acts strictly as a semantic proposal engine, while the harness applies hard-coded validation gates, sandboxed side-state rollbacks, and non-autoregressive decision filters. By embedding verification natively into the runtime, code correctness ceases to be an emergent probabilistic property of prompt phrasing and becomes a structural invariant enforced by code.

Comparative Architectural Analysis of Modern Coding Agents

To engineer an optimal deterministic harness, existing paradigms across open-source frameworks and enterprise platforms must be systematically evaluated. These implementations illustrate divergent philosophies regarding isolation boundaries, context window management,

tool invocation policies, and error recovery.

Comparative Framework Assessment

System	Primary Language & Runtime	Sandboxing & Execution Isolation	Deterministic Gating & Tool Verification	Context Management & Compaction	Architectural Vulnerabilities & Failure Modes
DeepSeek-Reasonix	Go (Static CGO-free binary)	Directory-level workspace sandbox; optional subprocess jail	Interceptor pattern via Extension Protocol v1/v2 (tool.before)	Structured prefix-cache alignment; stale tool output pruning	DeepSeek API coupling; lacks embedded AST or language server feedback loops
Open Interpreter	Rust core with Python/Shell runtime bindings	Unsandboxed by default; optional Docker and basic Semgrep scanning	Voluntary tool execution; interactive user permission confirmation	Ephemeral session state; lacks semantic compaction or persistent indexing	High vulnerability to shell injection; frequent breaking changes
Codewhale	Rust (Native binary, asynchronous Tokio engine)	Multi-platform OS isolation: macOS Seatbelt, Linux Landlock, seccomp, Bubblewrap	Constitutional authority ranking; embedded LSP diagnostics; side-git turn rollbacks	Persistent goal loop; durable task queue and session branching	Reasoning budget exhaustion on large diff reviews; high token latency
Qwen-Code	TypeScript / Node.js CLI	Process-level execution; basic directory path constraints	Pre-commit hook integration; voluntary built-in terminal tools	Standard context window trimming; @file manual context injection	Tool-calling breakdown on 30B/35B models without rigid grammar constraints
Claude Code	TypeScript / Bun runtime	Tiered permission engine (Ask, Plan, Auto, Bypass) with pattern rules	Defensive prompt contracts; mandatory read-before-write prerequisite gates	Multi-stage compaction: Tool Result Budget, Snip Compact, Microcompact, Auto-Compact	Heavy token overhead; expensive hosted API dependency; opaque heuristics
Google Antigravity	Desktop GUI / Go CLI / Gemini cloud runtime	Cloud VM or local workspace proxy with policy enforcement	Visual Artifact review; asynchronous subagent status verification	Enterprise audit logging; centralized repository indexing	Excessive token burn; high abstraction disconnects developer from concrete diffs

System	Primary Language & Runtime	Sandboxing & Execution Isolation	Deterministic Gating & Tool Verification	Context Management & Compaction	Architectural Vulnerabilities & Failure Modes
OpenCode	TypeScript / Go architecture	Managed cache repositories; local process isolation	Extension lifecycle hooks; OAuth-gated execution policies	Pluggable documentation indexing; read-only subagent context forks	Weak compiler integration; relies on model-directed bash calls for diagnostics

Architectural Syntheses and Incorporations

Codewhale

Codewhale demonstrates a critical architectural pattern: the shadow repository state machine implemented via "side-git". Rather than applying unverified model modifications directly to the developer's working directory or creating noisy speculative commits in the primary Git branch, Codewhale maintains an isolated external Git directory (.codewhale/side-git). Snapshots are taken before and after every turn, enabling immediate /restore rollback if compilation fails or regressions are detected.

Furthermore, Codewhale introduces a formal Constitutional hierarchy governing instruction authority: live evidence (tool and compiler output) outranks user instructions, which in turn outrank stale repository memories. Its integration of kernel-level isolation primitives—macOS Seatbelt, Linux Landlock, seccomp filters, and Bubblewrap—provides a blueprint for preventing rogue shell operations during autonomous execution.

Claude Code

Anthropic's architecture emphasizes defensive operational contracts and context preservation. The system prompt enforces tool prerequisites through structural constraints: the file editing tool will reject any modification if the target file has not been explicitly inspected during the active session. This prevents the model from attempting blind writes based on outdated training weights or assumptions.

Claude Code's context compaction pipeline systematically mitigates context window degradation across long sessions:

- *Tool Result Budget*: Enforces a hard byte and line ceiling on tool outputs, truncating long search or build traces to preserve attention bandwidth.
- *Snip Compact*: Drops historical, non-essential user-assistant intermediate turns while keeping terminal anchors intact.
- *Microcompact*: Selectively strips the verbose payloads of past tool responses (such as file reads) once subsequent operations have completed, preserving cache-stable input anchors.
- *Auto-Compact*: Executes an asynchronous, structured summarization pass once context consumption exceeds a defined threshold.

DeepSeek-Reasonix

Reasonix is engineered around prefix-cache stability, adhering to a "leave it running" operational philosophy. In modern inference engines, token evaluation costs and time-to-first-token latencies drop substantially when prompt prefixes remain static across requests. Reasonix achieves this by isolating dynamic context at the end of the prompt sequence, preventing early-token mutations that invalidate GPU Key-Value (KV) caches. It also incorporates an Extension Protocol supporting event interceptors (`tool.before`), which allows external binaries to inspect, mutate, or abort tool calls deterministically before execution.

Qwen-Code and OpenCode

Both engines highlight the utility of dividing cognitive labor through specialized subagents, such as dedicated documentation readers and test runners. However, both systems also expose the fragility of unconstrained JSON tool calling on mid-sized models. When running models around 30B to 35B parameters, unconstrained JSON generation frequently fails due to missing escape characters, malformed schemas, or conversational commentary prepended to the tool call. A robust harness must enforce constrained grammar decoding at the sampling layer.

Foundations and Mechanics of JEV and System 1 Models

A major architectural bottleneck in multi-agent coding systems is the reliance on massive autoregressive models for simple operational decisions, such as triage, risk categorization, and validation checks. Renting or hosting a high-parameter generative model to perform binary classification or single-parameter extraction introduces high latency, excessive token consumption, and non-deterministic formatting failures.

Cognitive Duality: System 1 versus System 2 in Software Control Planes

The conceptual foundation introduced by TypeSafe AI references Daniel Kahneman's division between cognitive modalities:

- *System 2 (Slow, Deliberate Reasoning)*: High-capacity autoregressive language models (such as Claude Opus, DeepSeek R1, or GPT-5-class architectures) that operate sequentially, generating token-by-token reasoning chains, narrative explanations, and code patches.
- *System 1 (Fast, Intuitive Decisions)*: Non-autoregressive or parallel-sampling classification models that ingest an arbitrary program state and immediately emit structured, typed decisions and confidence distributions without conversational text generation.

Standard LLMs aligned via Reinforcement Learning from Human Feedback (RLHF) are tuned to satisfy human conversational expectations, leading to mode dropping, verbosity, and uncalibrated overconfidence. When forced to evaluate an internal code diff, an RLHF-tuned model often generates persuasive natural language explanations justifying why broken code is acceptable. In contrast, a System 1 model discards freeform text generation entirely. It maps program state directly to typed data structures, returning discrete choices and calibrated probabilities in a single forward evaluation.

Operational Primitives and Mechanics of Jev

TypeSafe AI's flagship implementation, Jev, operates as a machine-native decision endpoint with end-to-end execution latencies between 70ms and 500ms, running roughly 40x to 200x faster than traditional frontier LLMs. It processes input states through three distinct primitives:

1. **Choice**: Categorical selection across a defined set of options (supporting cardinalities up to 255). $\text{Choice}(S, \{C_1, C_2, \dots, C_k\}) \rightarrow (C^*, \mathbf{P}, \gamma)$ where S represents the program state, C^* is the selected candidate, $\mathbf{P} = [p_1, p_2, \dots, p_k]$ is the probability distribution across all valid options ($\sum p_i = 1$), and $\gamma \in [0, 1]$ represents the calibrated decision confidence.
2. **Score**: Continuous or ordinal evaluation against an explicit rubric. $\text{Score}_{\text{start_span}}^{\text{end_span}}(S, \text{Rubric}) \rightarrow (s, \mathbf{P}_{\text{scale}}, \gamma)$ where s denotes the assigned scalar level, $\mathbf{P}_{\text{scale}}$ represents the distribution across score buckets, and γ is the associated confidence.
3. **Noul**: Epistemic evaluation of a binary proposition. $\text{Noul}(S, \phi) \rightarrow p(\phi = \text{True}) \in [0, 1]$ where p represents the probability that condition ϕ holds true within state S .

Mathematical Guarantees and Training Methodology

Jev achieves schema compliance by construction. Because its internal sampling engine evaluates logits strictly across pre-declared categorical heads or grammar states, the structural type error rate is identically 0%. The model cannot emit unparseable syntax, unbalanced brackets, or unlisted field names because invalid token paths are mathematically unreachable in the decoding graph.

The underlying training architecture uses Reinforcement Learning for Calibrated Decisions (RLCD). Rather than optimizing for human conversational preference, RLCD penalizes uncalibrated predictions using proper scoring rules (such as Brier score minimization or cross-entropy against empirical outcomes). Consequently, Jev's reported confidence γ possesses formal calibration:

$$P(C^* = C_{\text{true}} \mid \gamma = \alpha) \approx \alpha$$

When the model reports a confidence of 0.95, the empirical error rate across operational trials converges to 5%. Conversely, when the entropy of the probability distribution $\mathbf{P}$ is high, the model signals that the provided state contains ambiguous or insufficient data to make a reliable determination.

Operational Patterns and Limitations

The integration of non-autoregressive decision primitives enables specific harness patterns:

- *Speculative Fan-Out*: A code state is ingested once, and dozens of independent verification queries (such as checking null safety, boundary checks, concurrency races, and naming conventions) are evaluated in parallel during a single pass with negligible latency impact.
- *Confidence-Gated Routing*: Instead of applying a static global heuristic, the harness assigns risk-weighted confidence thresholds to actions. Low-risk modifications (such as updating comments) proceed autonomously at $\gamma \geq 0.75$, whereas modifications

to authentication logic or core build configs require $\gamma \geq 0.98$ or route to human review.

- *Composite Scoring*: Multi-factor assessments (such as code health) are decomposed into discrete, orthogonal criteria evaluated independently and combined via deterministic weighting formulas in code, preventing holistic reasoning bias.

Jev also exhibits known failure modes documented by TypeSafe AI:

- *Literal Bias*: The model evaluates syntax strictly as written and cannot extrapolate unstated human intentions or resolve informal ambiguities.
- *Arithmetic and Spatial Incompetence*: The architecture cannot perform numerical calculations, count tokens or lines of code, or evaluate relative dates.
- *Context Saturation*: Large, noisy context inputs degrade performance; the model cannot synthesize multi-hop logical inferences spread across disparate context fragments.
- *Prompt Injection Susceptibility*: Adversarial instructions embedded directly within source code comments or strings can skew categorical probabilities if ingested without sanitization.

Open-Source Implementations and Local Approximations

Because TypeSafe AI's Jev is a proprietary hosted API, open-source engineering efforts have developed functional local equivalents:

1. **Direct Logit Readout (TheoLeeCJ/openjev)**: Designed for local deployment on hardware such as NVIDIA RTX 3090/4090 or Apple Silicon, this approach uses standard open models (such as Qwen 3.5 4B or 35B). The input state is processed once through the model's transformer layers to generate the prompt KV cache. The context is then branched across target questions. Instead of sampling tokens autoregressively, the harness directly reads the raw, unnormalized logits from the vocabulary projection layer for the specific token IDs corresponding to valid answers (such as "A", "B", "C", or "true", "false"). Applying a localized softmax over this narrow sub-vocabulary yields instantaneous decision probabilities without token generation latency.
2. **NLI Cross-Encoder Backbones (AlexWortega/openjev)**: This paradigm frames System 1 verification as a Natural Language Inference (NLI) problem. By fine-tuning Qwen 3.5 variants (including 4B dense and 35B Mixture-of-Experts backbones) on cross-encoder tasks, the source code state is supplied as the *premise* and the verification condition as the *hypothesis*. The classification head outputs logits across three discrete labels: entailment, contradiction, and neutral. In this setup, argmax entailment represents verification approval, executing in a single forward evaluation pass.
3. **Masked Language Modeling Heads (ModernBERT / Latent MLP Heads)**: This architecture combines bidirectional encoder representations (such as ModernBERT-large) with task-specific multi-layer perceptron (MLP) heads. Pre-defined [MASK] tokens inserted into the schema template are resolved in parallel through a single forward pass, achieving decision latencies below 40ms on consumer GPUs.

Decision Paradigm Comparison

Metric / Dimension	Traditional Autoregressive LLM (System 2)	TypeSafe AI Jev (System 1)	Local Logit-Readout Approximation (openjev)
Output Type	Unstructured text string or parsed JSON	Native typed values, vectors, and scalars	Constrained logits mapped to discrete schema keys
Decoding Mechanism	Sequential autoregressive token generation	Non-autoregressive parallel sampler	Single-token forward pass logit extraction
Execution Latency	3,000ms – 300,000ms	70ms – 500ms	35ms – 250ms (local GPU dependent)
Type Invariant Error	0.5% – 45.0% schema invalidity	0.0% (Guaranteed by construction)	0.0% (Masked logit extraction)
Confidence Metric	Overconfident verbal assertions	RLCD-calibrated probability score	Softmax over target token logit slice
Compute Cost	\$0.20 – \$10.00 / MTok	\$0.042 / MTok input; outputs free	Local electricity / amortized hardware

Hard-Coded Deterministic Gating Engine

To force a 35B model to produce reliable code, the harness must eliminate all voluntary verification steps. The runtime acts as a gatekeeper: any patch generated by the model is staged in memory, analyzed through concrete syntax tree parsers, compiled via Language Server Protocol (LSP) daemons, and checked against static analysis security rules before disk serialization or commit staging can occur.

Bundled Static Analysis and Verification Tooling

The harness bundles high-speed, native verification tooling, avoiding external dependencies or project-specific package configurations:

Tooling Class	Bundled Utility	Integration Mechanism	Target Languages / Artifacts	Verification Scope
AST Parser	tree-sitter	In-memory C/Rust dynamic bindings	Polyglot (Rust, TS/JS, Python, Go, C/C++)	Structural grammar validity; concrete syntax tree verification; syntax error interception.
Type & Compiler Daemon	Headless LSP daemons (rust-analyzer, pyright, vtls, gopls, clangd)	Bidirectional JSON-RPC over background stdio pipes	Rust, Python, TypeScript, Go, C/C++	Cross-file type correctness; interface implementation; import verification; compilation diagnostics.
Structural Pattern Linter	ast-grep	In-process native binary / Rust crate	Polyglot	Syntactic anti-patterns;

Tooling Class	Bundled Utility	Integration Mechanism	Target Languages / Artifacts	Verification Scope
		bindings		unwrap safety; forbidden standard library invocations; architectural boundaries.
Security & Taint Scanner	semgrep-core	Standalone CLI invocation against shadow tree	Polyglot	SAST rules; CWE vulnerability detection; taint tracking; hardcoded secret discovery.
High-Speed Linter/Formatter	ruff (Python) & biome (TS/JS)	Native compiled execution	Python, TypeScript, JavaScript, JSON, CSS	Style enforcement; import sorting; unused variable detection; syntax micro-optimizations.

Deterministic Gating Pipeline Mechanics

When an agent synthesizes a patch, the change passes through four sequential validation stages within the harness:

Stage 1: Concrete Syntax Tree (CST) Validation via Tree-sitter

Before writing to disk or allocating compiler resources, the proposed diff is applied to an in-memory representation of the target file. The resulting virtual file is parsed using the relevant tree-sitter language grammar. The parser inspects the syntax tree for any `ERROR` or `MISSING` nodes. If an unclosed brace, malformed string literal, or broken statement is detected, the patch is rejected immediately. The raw syntax error and its exact line and column coordinates are packaged into an error frame, terminating the turn before any filesystem mutations occur.

Stage 2: Sandboxed Patch Application via Side-Git Shadow Worktree

If the patch is syntactically valid, the harness mirrors the active workspace into an isolated shadow directory managed by an independent Git instance (`.harness/side-git`). The patch is applied to the shadow worktree using atomic patch utilities. If the patch fails to apply cleanly due to line offset drift or stale hunk context, the harness rejects the operation, forces the model to refresh its file read buffer, and preserves the user's working directory intact.

Stage 3: Language Server Protocol (LSP) Diagnostic Verification

The harness maintains persistent, background-managed LSP daemon processes communicating over JSON-RPC stdio pipes. Following a clean patch application in the shadow worktree, the harness emits a `textDocument/didOpen` or `textDocument/didChange` notification to the appropriate language server (e.g., `rust-analyzer` for Rust, `pyright` for Python, `vtls` for

TypeScript). The harness blocks until the server publishes diagnostics via `textDocument/publishDiagnostics`.

- Diagnostics are filtered by `DiagnosticSeverity.Error` (severity level 1).
- If compiler or type errors are reported, the shadow worktree is reverted to the pre-turn snapshot via `side-git`.
- The diagnostic payload is extracted and converted into a structured error frame.

Stage 4: Structural Linters and Static Application Security Testing (SAST)

Once the code compiles cleanly, the harness executes native static analysis checks against the shadow worktree using embedded `ast-grep` and `semgrep-core` binaries. These tools enforce structural constraints that type checkers miss: preventing insecure SQL formatting, blocking unhandled thread joins, and verifying that internal architectural boundaries are respected. Simultaneously, high-speed linters (`ruff` for Python, `biome` for TypeScript) format the code automatically, eliminating stylistic arguments and import errors before review.

Diagnostic Error Frame Construction

Language models around 35B parameters frequently degrade when exposed to verbose, multi-kilobyte terminal error traces, often fixating on incidental warnings rather than root causes. The harness intercepts raw tool output, strips terminal escape codes and redundant call stacks, and formats the diagnostic into a structured feedback frame:

```
[DETERMINISTIC_GATE_VIOLATION: COMPILER_DIAGNOSTIC] File:
src/services/auth_provider.py Line: 84, Column: 22 LSP Provider: pyright
(reportGeneralTypeIssues) Severity: ERROR Message: Argument of type "str | None" cannot be
assigned to parameter "token" of type "str" in function "verify_jwt". Context: 82 | user_session =
self.session_cache.get(session_id) 83 | raw_token = user_session.get_token()
84 | return self.verify_jwt(raw_token) ^^^^^^^^^^^ Resolution Requirement: Add an explicit guard
verifying that "raw_token" is not None before invocation, or raise an AuthException. Instruction:
Emit the corrected replacement hunk for lines 82-85. Do not include narrative conversational
text.
```

By presenting an isolated, actionable diagnostic containing source context, the exact failure location, and a concrete resolution requirement, the model converges on a working fix without losing reasoning context.

Adversarial Multi-Agent Review Architecture and Feedback Loops

Self-correction within a single LLM context is inherently prone to confirmation bias: a model that introduces a logical flaw or security hole is likely to overlook that same flaw during a self-reflective pass. To catch bugs reliably before code is committed to version control, the harness implements an adversarial multi-agent review architecture governed by hard-coded execution loops.

Generator-Auditor Structural Separation

The architecture separates generation from critical evaluation by assigning them to two distinct

operational profiles:

1. **The Generator Agent:** Operates under a system prompt tailored for task decomposition, codebase search, and implementation. The Generator is permitted to call search tools (FileRead, Grep, Glob) and synthesize proposed edits. It is restricted from issuing Git commits, merging branches, or declaring tasks complete.
2. **The Auditor Agent:** Operates under an adversarial system prompt designed to locate logical regressions, unhandled edge cases, and boundary vulnerabilities. The Auditor has read-only access to the codebase and shadow diffs; it is physically blocked by the harness from modifying production code files. Its only available outputs are structured critique reports and standalone test suites designed to expose implementation flaws.

The Two-Pass Pre-Commit Verification Lifecycle

Every patch that clears the deterministic compiler gates must pass through two successive adversarial review cycles before a Git commit can be created:

Pass 1: Semantic Invariant and Logic Review

The Auditor receives the user's initial prompt, the project specification, and the candidate patch applied to the shadow tree. It evaluates the change against three criteria:

- *Specification Adherence:* Does the diff implement all requirements without altering unrelated application logic or public API contracts?
- *Resource Management:* Are file handles, database connections, and memory allocations correctly cleaned up across all error and panic branches?
- *Edge-Case Exhaustion:* Does the logic account for null inputs, empty collections, dropped network packets, and concurrent access?

If the Auditor identifies a flaw, it emits an objection frame detailing the failing scenario. The harness passes this report back to the Generator for revision. If the Generator cannot resolve the objection, the loop halts.

Pass 2: Empirical Test Synthesis and Security Verification

To prevent reviews from devolving into subjective debates over code style, Pass 2 requires empirical verification.

- The Auditor is tasked with generating a concrete, executable unit or integration test designed specifically to break the Generator's code based on the identified risks.
- The harness writes this test file to an isolated test directory within the sandboxed shadow worktree.
- The harness executes the test suite using native test runners (e.g., cargo test, pytest, vitest) inside an isolated execution sandbox (Bubblewrap or Docker).
- *Empirical Gate:* If the Auditor's test fails (demonstrating a bug in the Generator's code), the patch is rejected. The test output and stack trace are injected into the Generator's context for remediation. If the test passes, the test itself is retained and staged alongside the patch as an enduring regression safeguard.

Mitigating Loop Stagnation and Adversarial Hallucinations

Adversarial multi-agent workflows can fall victim to *cyclic stagnation*, where agents argue

indefinitely over subjective preferences, exhausting token budgets and stalling execution. The harness mitigates this through hard-coded controls:

- **Strict Two-Iteration Turn Cap:** The review loop allows a maximum of two corrective cycles between Generator and Auditor. If the patch fails to achieve clean compilation and Auditor approval by the second pass, the harness aborts the operation, rolls back the shadow worktree via side-git, and escalates the issue to the human operator with a failure summary.
- **Evidence Supremacy Rule:** The harness enforces Codewhale's constitutional principle: *evidence outranks narration*. An Auditor cannot block a patch using speculative natural language. An objection is only recognized by the harness if it is paired with an LSP diagnostic error, a static analysis violation, or an executable test case that fails deterministically.
- **Non-Autoregressive System 1 Arbitration:** If the Generator and Auditor reach an impasse regarding whether a specification requirement has been met, the dispute is resolved by an embedded System 1 classification model (using the Jev-style Noul primitive). Ingesting the specification, the diff, and the objection, the System 1 arbiter evaluates $P(\text{Defect Present} \mid \text{Spec}, \text{Diff})$. If the probability is below 0.15, the Auditor's objection is overruled, terminating the debate.

Harness Engineering for Local Execution on Qwen 3.5 35B

Optimizing a harness for a local 35B-parameter model requires configuring the inference runtime to ensure throughput, manage context limits, and guarantee schema compliance.

Local Inference Engine Configuration and Memory Budgeting

The harness interfaces with **vLLM** or **SGLang** through local Unix Domain Sockets (`/run/harness/vllm.sock`), avoiding network stack overhead and eliminating telemetry leaks:

- **Quantization Selection:** In full 16-bit precision (FP16/BF16), a 35B model requires ~70 GB of VRAM, exceeding standard consumer hardware. The harness targets AWQ or EXL2 4-bit quantization, which compresses the model weights to ~20 GB. This allows the model to run on a single 24GB GPU (such as an NVIDIA RTX 3090, 4090, or 5090) or unified Apple Silicon (32GB+), leaving 2GB to 4GB of VRAM dedicated to the Key-Value (KV) cache.
- **Active Context Ceiling:** Although Qwen 3.5 supports extended context windows (up to 32k or 128k tokens), attention quality and instruction following degrade significantly as the context fills. The harness enforces a strict active context limit of 16,384 tokens, maintaining attention precision and suppressing context rot.

Prefix-Cache Layout and KV Cache Protection

To minimize token generation latency across iterative agent turns, the harness structures its prompt into three distinct caching tiers, ensuring high Key-Value (KV) cache reuse across turns:

1. **Zone 1: Immutable System Core (Static Prefix):** Contains base behavioral constraints, tool schemas, compiler error formatting rules, and harness policies. This block is completely frozen; not a single character or token position changes throughout the session. Inference engines achieve a 100% KV cache hit rate across this segment, eliminating prompt evaluation latency for the base prompt.
2. **Zone 2: Workspace Architectural Topology (Semi-Static Buffer):** Contains the file tree, package manifests, build tool configurations, and project rules. This block is updated only when files are created, renamed, or deleted. During standard edit-compile cycles on existing files, this zone remains stable in the KV cache.
3. **Zone 3: Working Turn Tail (Dynamic Context Window):** Contains active task descriptions, recent file diffs, and the latest compiler feedback frames. This tail segment is aggressively managed using Claude Code-style microcompaction: once a file edit is confirmed, verbose intermediate file dumps are replaced with concise patch summaries, preserving cache alignment for earlier zones.

Grammar-Guided Constrained Decoding

Mid-sized models can struggle with unconstrained structured generation, occasionally emitting leading conversational filler (e.g., *"Here is the tool call you requested..."*) or generating invalid JSON syntax under high context loads. The harness eliminates this failure mode by integrating **Grammar-Guided Constrained Decoding** (via GBNF grammars or Outlines/vLLM guided decoding engines) directly into the inference loop.

When the model is prompted to emit a tool call or a code patch, the harness supplies a formal Context-Free Grammar (CFG). The inference engine masks the logit probabilities at each step, making tokens that violate the grammar structurally impossible to sample. The model is constrained to valid tool names, strictly typed arguments, and valid string termination, ensuring zero JSON parsing errors during tool execution.

End-to-End System Execution Lifecycle

The following lifecycle details each operational phase from user intent ingestion to the final verified Git commit, outlining the executing component, active deterministic verification gates, and automated rollback triggers:

Step	Lifecycle Phase	Executing Component	Deterministic Gate / Verification Mechanism	Rollback Trigger & Recovery Action
1	Task Triage & Risk Categorization	Embedded System 1 Model (openjev Logit-Readout)	Evaluates task complexity; assigns risk level; checks for prompt injection in user input.	If prompt injection is detected or state is ambiguous, task is aborted and escalated to user.
2	Exploration & Context Assembly	Generator Agent (Plan Posture)	Read-only tools (FileRead, Grep, Glob). Filesystem write APIs are locked.	Attempts to access files outside workspace bounds trigger an immediate tool

Step	Lifecycle Phase	Executing Component	Deterministic Gate / Verification Mechanism	Rollback Trigger & Recovery Action
				rejection.
3	Patch Synthesis	Generator Agent (Agent Posture)	Grammar-constrained decoding enforces valid diff schema. Mandatory read-before-write validation.	Syntactic or schema violations are blocked at the token sampling layer.
4	Concrete Syntax Tree Check	In-Process tree-sitter Parser	Parses modified file in memory; verifies that no syntax errors or incomplete nodes exist.	Syntax failure halts cycle; error frame returned to Generator; disk remains untouched.
5	Sandboxed State Staging	Harness Isolation Engine	Clones current workspace into .harness/side-git; applies patch to shadow tree.	Patch application collision or hunk offset failure resets shadow tree to baseline.
6	Compiler & Type Verification	Headless LSP Daemon (rust-analyzer, pyright)	In-memory buffer sync; queries publishDiagnostics for errors (Severity 1).	Diagnostics parsed into structured error frame; shadow tree reverted via side-git.
7	Structural SAST & Lint Gate	Embedded ast-grep, semgrep, ruff, biome	Scans diff against AST pattern rules, security vulnerabilities, and formatting standards.	Rule violations trigger side-git rollback; diagnostic returned to Generator for remediation.
8	Adversarial Pass 1: Semantic Review	Auditor Agent (Adversarial Persona)	Evaluates diff against project specifications, resource leaks, and edge-case handling.	Logical objections packaged into critique frame; Generator granted one remediation turn.
9	Adversarial Pass 2: Empirical QA	Auditor Agent & Sandboxed Test Runner	Auditor synthesizes an executable reproducing unit test; runs inside container sandbox.	Test failure indicates bug; test logs returned to Generator; turn cap enforced.

Step	Lifecycle Phase	Executing Component	Deterministic Gate / Verification Mechanism	Rollback Trigger & Recovery Action
10	Consensus Arbitration Gate	Embedded System 1 Arbiter (Noul / Score)	Evaluates empirical pass status: commit allowed only if $\gamma_{\text{pass}} \geq 0.90$ and $p(\text{defect}) \leq 0.05$.	If consensus fails after two cycles, session halts; workspace rolls back via /restore.
11	Repository Commit & Session Seal	Harness Version Control Engine	Merges validated shadow tree diff into main workspace Git index; creates verified commit.	None (Atomic operation).

Architectural Synthesis and Recommendations

Enforcing reliable, production-grade software engineering from mid-sized models such as Qwen 3.5 35B requires eliminating reliance on voluntary model compliance. Mid-sized models do not fail because they lack programming knowledge; they fail because standard harnesses overwhelm them with administrative coordination, allow context degradation, and expect them to self-correct without deterministic feedback.

By constructing a deterministic harness that combines:

- Non-autoregressive System 1 decision primitives (via Jev-style direct logit readouts or NLI cross-encoders) for low-latency routing, triage, and consensus arbitration,
- Hard-coded compiler, AST, and Language Server Protocol gates embedded directly into the execution control plane,
- Shadow filesystem sandboxing via external side-git snapshots for non-destructive rollbacks,
- An adversarial Generator-Auditor architecture governed by empirical test synthesis rather than subjective debate, and
- Grammar-constrained decoding paired with cache-aligned prompt layouts,

the engineering harness transforms an open-weights 35B model into an autonomous development engine. Correctness is no longer left to chance—it is enforced by the compiler architecture of the harness itself.

Works cited

1. AlexWortega/openjev - Hugging Face, <https://huggingface.co/AlexWortega/openjev>
2. Tool calling does not work with local model qwen3-30b-a3b #176, <https://github.com/QwenLM/qwen-code/issues/176>
3. Claude Code Leak: 7 Prompt Engineering Techniques ... - North Park, <https://northparkgroup.co/blog/claude-code-leak-prompt-engineering-rules>
4. ContextCov: Bridging the Gap Between Developer Intent and ... - arXiv, <https://arxiv.org/html/2603.00822v2>
5. codewhale - X-CMD, <https://www.x-cmd.com/install/codewhale/>
6. claude-code/Skill.md at

main · codeaashu/claude-code - GitHub,
<https://github.com/codeaashu/claude-code/blob/main/Skill.md> 7. OpenClaw vs Open Interpreter:
The Ultimate Guide to Local AI Agents,
<https://skywork.ai/skypage/en/openclaw-open-interpreter-local-ai-agents/205300673688898355>
2 8. deepseek-core - CodeWhale - Lib.rs, <https://lib.rs/crates/deepseek-core> 9. TypeSafe AI's
Jev: What "System One Models" Actually Are, <https://www.truefoundry.com/blog/typesafe-ai-jev>
10. Codewhale/docs/CONFIGURATION.md at main - GitHub,
<https://github.com/Hmbown/CodeWhale/blob/main/docs/CONFIGURATION.md> 11. Claude
Code Architecture Analysis - Bits, Bytes and Neural Networks,
[https://bits-bytes-nn.github.io/insights/agent-ai/2026/03/31/claude-code-architecture-analysis.h](https://bits-bytes-nn.github.io/insights/agent-ai/2026/03/31/claude-code-architecture-analysis.html)
tml 12. DeepSeek Reasonix: Builder Guide - Verdent AI,
<https://www.verdent.ai/guides/deepseek-reasonix> 13. Issues · esengine/DeepSeek-Reasonix -
GitHub, <https://github.com/esengine/reasonix/labels/good%20first%20issue> 14. Qwen Code is
an open-source AI agent, <https://qwen.ai/qwencode> 15. Agents - OpenCode,
<https://opencode.ai/docs/agents/> 16. esengine/DeepSeek-Reasonix: DeepSeek-native AI coding
agent, <https://github.com/esengine/deepseek-reasonix> 17. Jev Explained: Typesafe AI's
Non-Autoregressive System-1 Model,
<https://www.mindstudio.ai/blog/jev-system-one-model-launch> 18. How to Use Jev: A practical
guide to TypeSafe's System One model,
<https://dev.to/valyuai/how-to-use-jev-a-practical-guide-to-typesafes-system-one-model-g5e> 19.
TypeSafe AI Releases Jev: A System One Model That Returns,
<https://www.marktechpost.com/2026/09/19/typesafe-ai-releases-jev/> 20. TypeSafe AI: Home,
<https://typesafe.ai/> 21. I read OpenJev and thought of an alternative solution using non,
<https://dev.classmethod.jp/en/articles/openjev-non-generative-ai-alternatives/> 22.
TheoLeeCJ/openjev — GitHub trending stats & insights | Trendshift,
<https://trendshift.io/repositories/241523> 23. OpenJev | Hacker News,
<https://news.ycombinator.com/item?id=49752041> 24. Openjev : r/LocalLLaMA - Reddit,
<https://www.reddit.com/r/LocalLLaMA/comments/1wib9kj/openjev/> 25. Made the horizontal
open-source model for Jev with RLCD ... - Reddit,
[https://www.reddit.com/r/LocalLLaMA/comments/1wjieap/made_the_horizontal_opensource_mo](https://www.reddit.com/r/LocalLLaMA/comments/1wjieap/made_the_horizontal_opensource_model_for_jev_with/)
del_for_jev_with/ 26. How aider scored SOTA 26.3% on SWE Bench Lite,
<https://aider.chat/2024/05/22/swe-bench-lite.html> 27. A curated list of static analysis (SAST)
tools and linters for all, <https://github.com/analysis-tools-dev/static-analysis> 28. The Top 13
Code Vulnerability Scanners in 2026 - Aikido Security,
<https://www.aikido.dev/blog/top-code-vulnerability-scanners> 29. Issues · Hmbown/Codewhale -
GitHub, <https://github.com/Hmbown/DeepSeek-TUI/issues> 30. Is Claude Code a Chatbot With a
Terminal Bolted On - Medium,
<https://medium.com/@krthiak/is-claude-code-a-chatbot-with-a-terminal-bolted-on-6f075a1b3bd4>
31. Hmbown/Codewhale - GitHub, <https://github.com/Hmbown/Codewhale> 32. a powerful
open-source coding agent + NO TELEMETRY FORK,
[https://www.reddit.com/r/LocalLLaMA/comments/1rar6md/qwen_code_a_powerful_opensource](https://www.reddit.com/r/LocalLLaMA/comments/1rar6md/qwen_code_a_powerful_opensource_coding_agent_no/)
_coding_agent_no/ 33. Codewhale/docs/RUNTIME_API.md at main - GitHub,
https://github.com/Hmbown/CodeWhale/blob/main/docs/RUNTIME_API.md